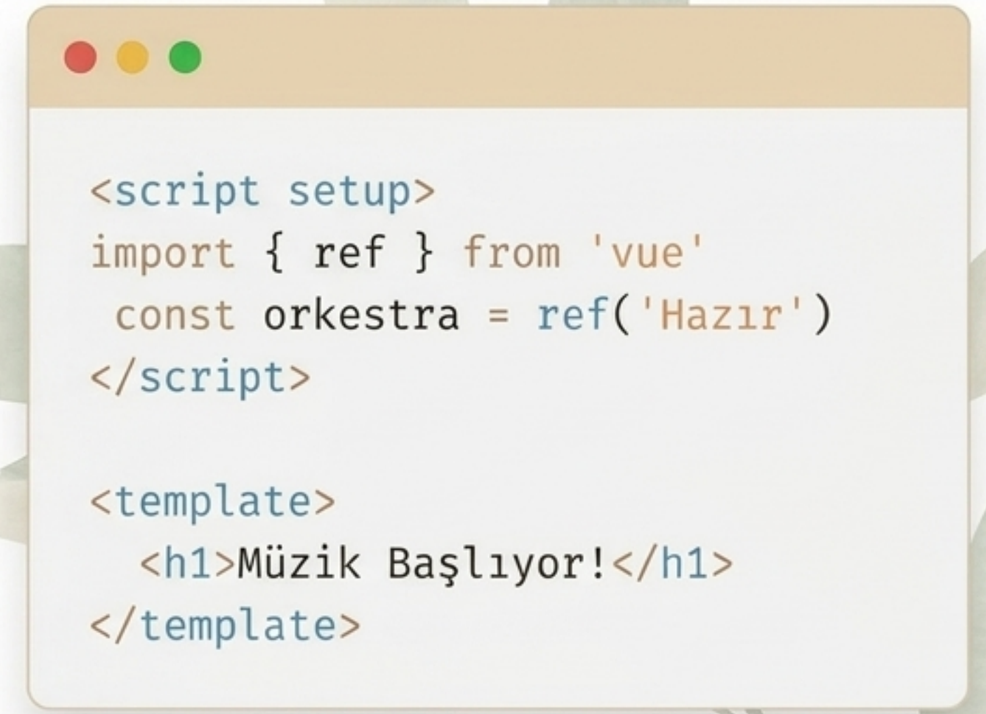
WEB PROGRAMLAMA I · VUE.JS ORKESTRASI

Vue.js Orkestrasını Yönetmek

Managing the Vue.js Orchestra



SCÜ Şarkışla UBYO | Web Programlama I

Uyarı: Bu İçerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokumantasyon kaynakları kendi orijinal lisanslarına tabidir. Hiçbir ticarî a

Gemini Notebook

A Revised Verision of Slide B

```
<h1 v-if="awesome">Vue harikadır!</h1>
```

Eğer 'awesome' ifadesi **gerçeğe uygun (truthy)** ise element yaratılır.

Bu gerçek bir koşullu render işlemidir. İfade false ise DOM'da bu elementin hiçbir izi kalmaz.

Benzer Örnekler:

```
<div v-if="isVisible">İçerik görünüyor</div>
```

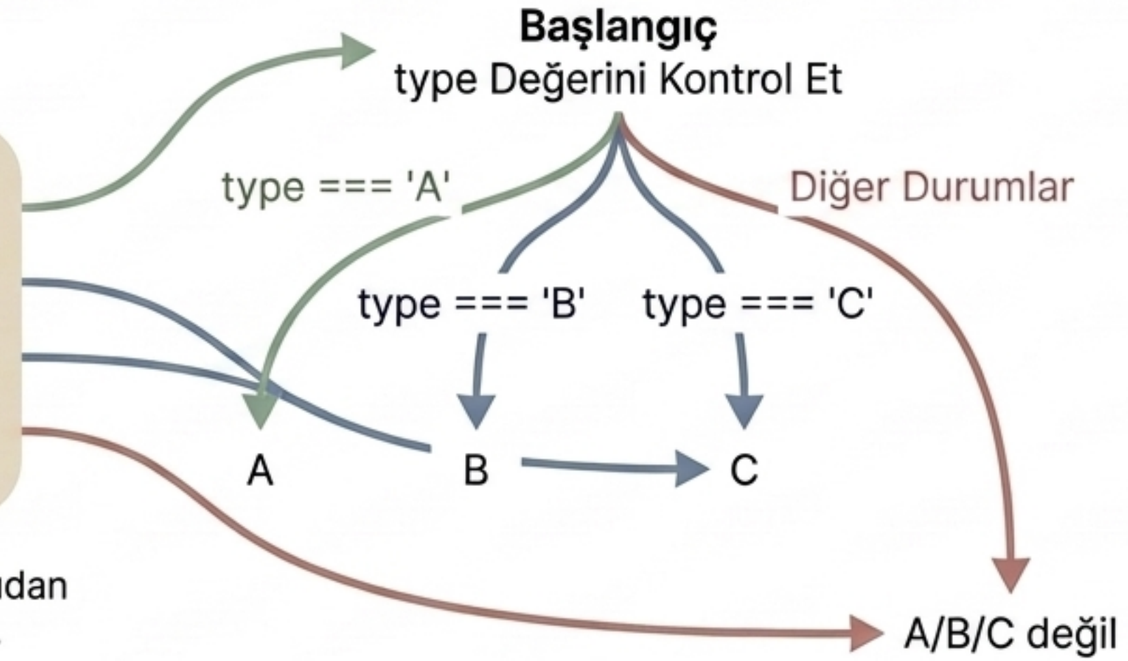
```
<button v-if="user.isLoggedIn">Çıkış Yap</button>
```

```
<span v-if="score > 10">Yüksek Skor!</span>
```

Gemini Notebook

Karmaşık Senaryolar İçin Alternatif Mantık Zincirleri

```
<div v-if="type === 'A'"> A </div>
<div v-else-if="type === 'B'"> B </div>
<div v-else-if="type === 'C'"> C </div>
<div v-else> A/B/C değil </div>
```



Zorunlu Kural: Bir **v-else** veya **v-else-if** öğesi, doğrudan bir önceki **v-if velse-if** öğesini hemen takip etmelidir. Aksi takdirde şablon derleyicisi tarafından tanınmaz.

Diğer Benzer Örnekler:

```
<div v-if="!user">Giriş Yapın</div>
<div v-else>Hoş Geldiniz!</div>
```

```
<div v-if="score > 90">Harika!</div>
<div v-else-if="score > 70">İyil</div>
<div v-else>Geliştirilebilir.</div>
```

```
<div v-if="status === 'pending'">Hazırlanıyor...</div>
<div v-else-if="status === 'shipped'">Kargoda</div>
<div v-else>Teslim Edildi</div>
```

Gemini Notebook

Görünmez Bir Sarıcı Olarak <template> Kullanımı

```
<template v-if="ok">
  <h1>Başlık</h1>
  <p>Paragraf 1</p>
  <p>Paragraf 2</p>
</template>
```

Tek koşula bağlı çoklu element.

<h1>Başlık</h1>

Paragraf 1

Paragraf 2

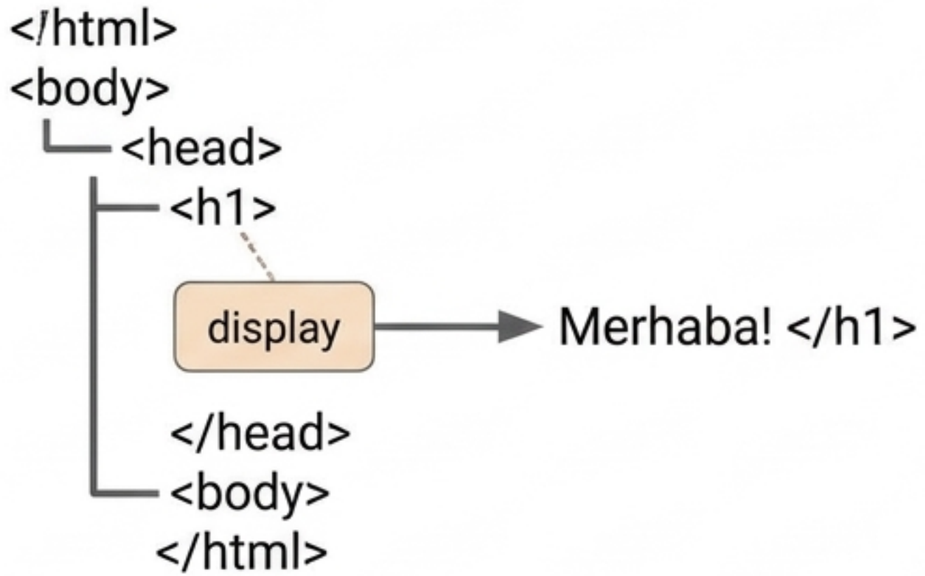
Yükleniyor...

Varsayılan durum (**v-else**) için grup.

Gemini Notebook

v-show Elementi Silmez Sadece CSS Üzerinden Gizler

```
<h1 v-show="ok">Merhaba!</h1>
```



Öğe her zaman render edilir ve DOM'da kalır.

Yalnızca CSS display özelliği (none) açılıp kapatılır.

Kısıtlamalar:

- Öğe pendirner.
- Öğe her zarramler ve DOM'da kalır.
- Yalnızca CSS display özelliği (none)
- Kasıızca dili kenrom, mawasi açılıp kapatılır.

Benzer Örnekler:

```
<p v-show="isVisible">Bu bir paragraftır.</p>
```

```
<div v-show="hasData">İçerik burada.</div>
```

Gemini Notebook

Mekanizma ve Maliyet Açısından v-if İle v-show Karşılaştırması

Özellik	v-if	v-show
Mekanizma	Gerçek Koşullu Render. Olay dinleyicileri ve alt bileşenler geçişlerde tamamen yok edilir ve yeniden oluşturulur.	Sadece CSS tabanlı geçiş (display: none).
İlk Yükleme	Tembeldir (Lazy). Koşul sağlanana kadar hiçbir işlem yapmaz.	Koşuldan bağımsız olarak her zaman render edilir.
Performans Maliyeti	Geçiş maliyeti (Toggle Cost) YÜKSEKTİR.	Başlangıç render maliyeti (Initial Cost) YÜKSEKTİR.
Benzer Örnekler	Bir binayı tamamen yıkıp yeniden inşa etmek (Gerçekten yok eder ve baştan yaratır).	Bir binanın önüne perde çekmek (Sadece görünürlüğünü gizler, bina yerinde durur).

Gemini Notebook

Performans Kuralı Çerçevesinde Doğru Direktifi Seçin

Bir öğeyi çok sık açıp kapatmanız gerekiyorsa...

→

v-show Kullanın
(Daha hızlı yanıt süresi avantaji sağlar)

Koşul çalışma zamanında nadiren değişecekse...

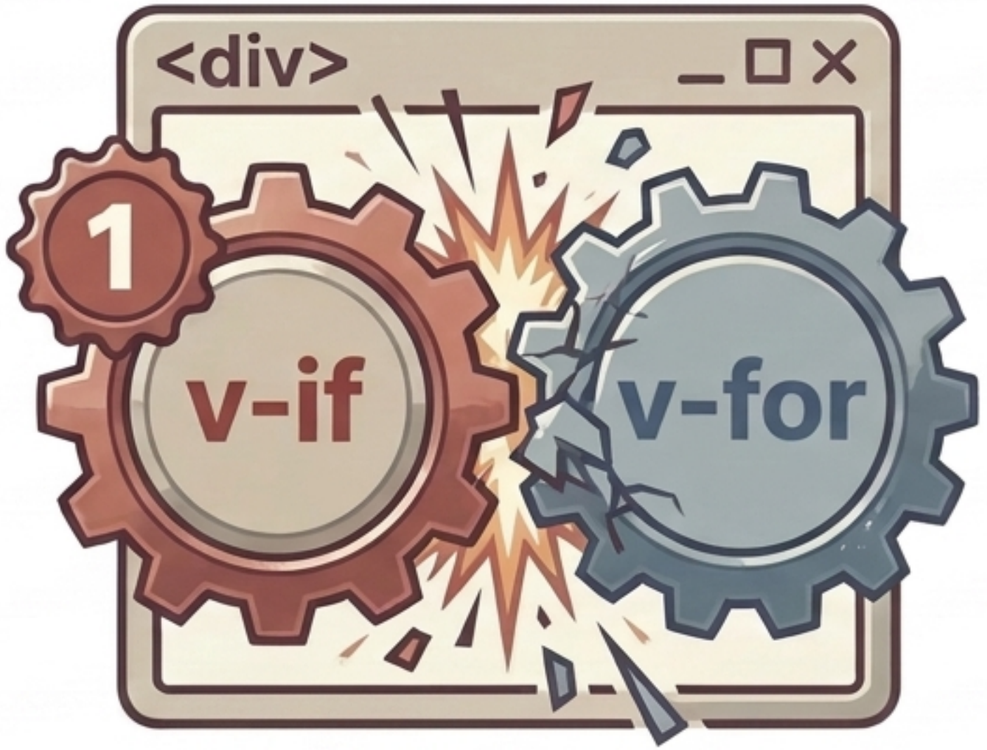
→

v-if Kullanın
(Düşük başlangıç maliyeti ve tembel yükleme avantajı sağlar)

Kullanıcı giriş yapmış mı, admin mi gibi sadece sayfa yük yüklenirken belirlenen durumlarda...

Gemini Notebook

Kritik Mimari Hata: Aynı Öğede v-if ve v-for Kullanımı



- Aynı öğe üzerinde kullanıldıklarında örtük öncelik kuralları devreye girer.
- v-if her zaman v-for'dan önce değerlendirilir.
- **Sonuç:** v-if koşulu henüz döngü başlamadığı için v-for kapsamındaki değişkenlere erişemez. Bu durum kaçınılmaz olarak bir derleme hatasıyla sonuçlanır.

Benzer Örnekler:

Hatalı Kullanım:

```
<ul>
  <li v-if="item.isActive" v-for="item in items"> {{ item.name }} </li>
</ul>
```

Doğru Kullanım (Seçenekler):

 <li v-for="item in activeItems"> {{ item.name }}   (activeItems hesaplanmış özelliği)

<li v-for="item in items"> {{ item.name

Gemini Notebook

2). Bölüm: Liste Render Etme (List Rendering

Dizileri arayüz elemanlarına dönüştürmenin en temel yolu.

```
const items = ref([
  { isim: 'Yedigöller Milli Parkı' },
  { isim: 'Salda Gölü' }
])
```

İsteğe bağlı dizin numarası (index) her zaman ikinci parametredir.

```
<li v-for="(item, index) in items">
  {{ index }} - {{ item.isim }}
</li>
```

0 - Yedigöller Milli Parkı

1 - Salda Gölü

Gemini Notebook

Döngüler sadece dizilerle sınırlı değildir.

Objects

```
<li v-for="(value, key, index) in bitkiSınıfı">
  {{ index }}. {{ key }}: {{ value }}
</li>
```

Yineleme sırası, **Object.values()** sonucuna dayanır.

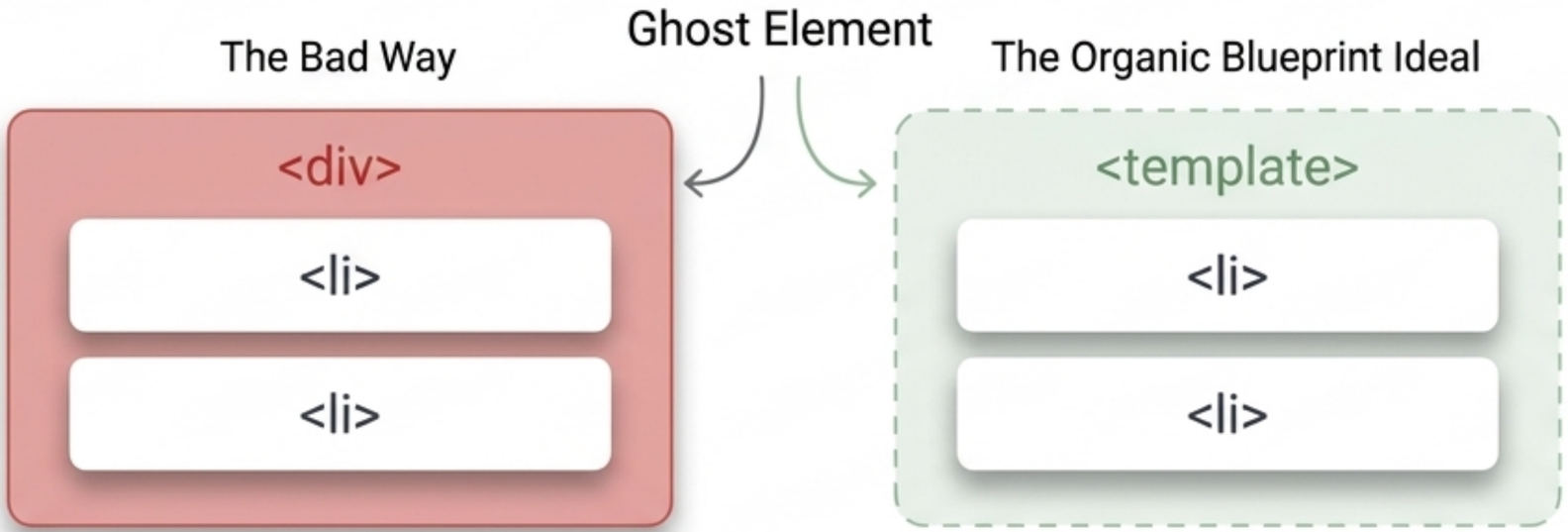
Ranges

Dikkat: Aralıklar 0'dan değil, 1'den başlar!

```
<span v-for="n in 10">{{ n }}</span>
```

Gemini Notebook

Gereksiz HTML sarmalayıcılarından kaçınmak için görünmez şablonlar kullanın.

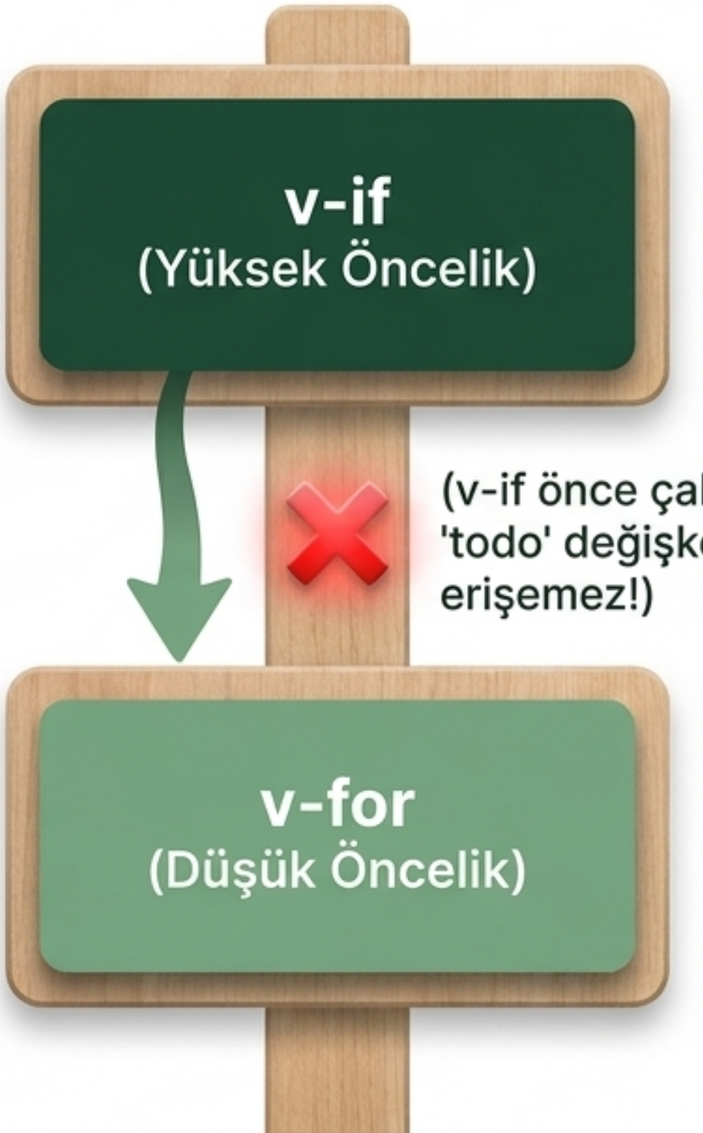



```
<ul>
  <template v-for="park in milliParklar">
    <li>{{ park.isim }}</li>
    <li class="ayirici" role="presentation"></li>
  </template>
</ul>
```

`<template>` etiketi HTML çıktısında görünmez. Birden fazla öğeyi gruplamak için mükemmel bir görünmez sarmalayıcıdır (wrapper).

Gemini Notebook

Aynı düğüm üzerinde v-if ve v-for kullanmak kesin bir kural ihlalidir.



Hatalı Kullanım

```
<li v-for="todo in todos"
  v-if="!todo.isComplete">
```

(Hata: todo henüz mevcut değil!)

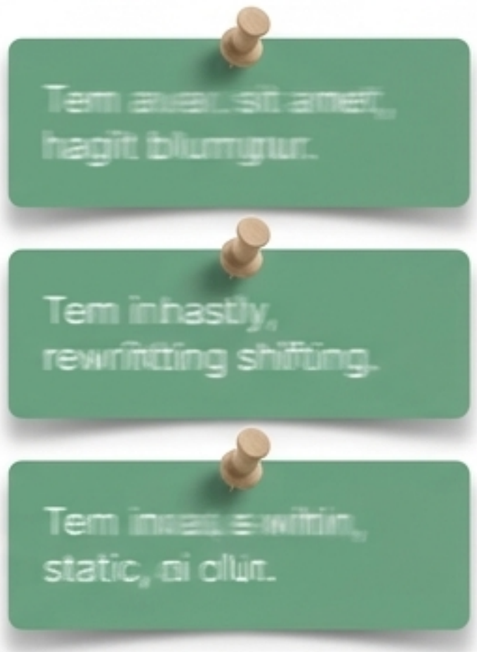
Doğru Çözüm

```
<template v-for="todo in todos">
  <li v-if="!todo.isComplete">...</li>
</template>
```

Gemini Notebook

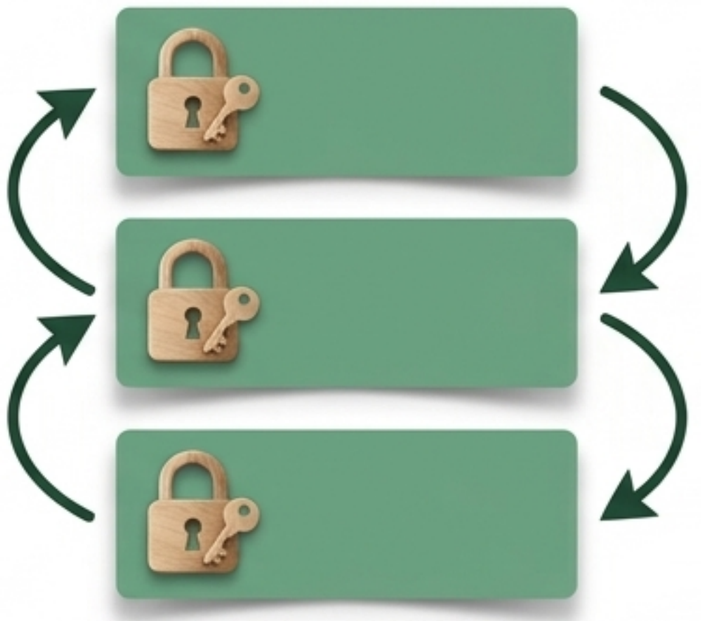
Vue'nun DOM güncellemelerini doğru takip edebilmesi için kimlik çapalarına ihtiyacı vardır.

Yerinde Yama (Without Key)



Geçici DOM durumları (formlar vb.) bozulur.

Kimlik Takibi (With Key)



```
<div v-for="item in items" :key="item.id">
```

En İyi Pratik

Benzersiz ve değişmez değerler kullanın (String veya Number). Asla nesneleri (objects) anahtar olarak kullanmayın.

Gemini Notebook

Vue'nun Sanal DOM motoru farklı dizi mutasyonlarına akıllıca tepki verir.

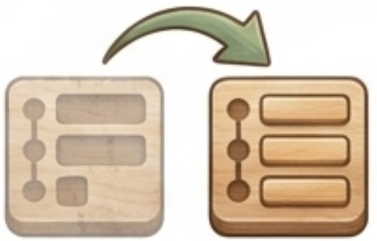
Mutasyon Metotları

Orijinal diziyi doğrudan değiştirir. Vue bu metotları sarar ve otomatik dinler.

`push()` `pop()` `shift()`

`splice()` `sort()` `reverse()`

Değiştirme Metotları



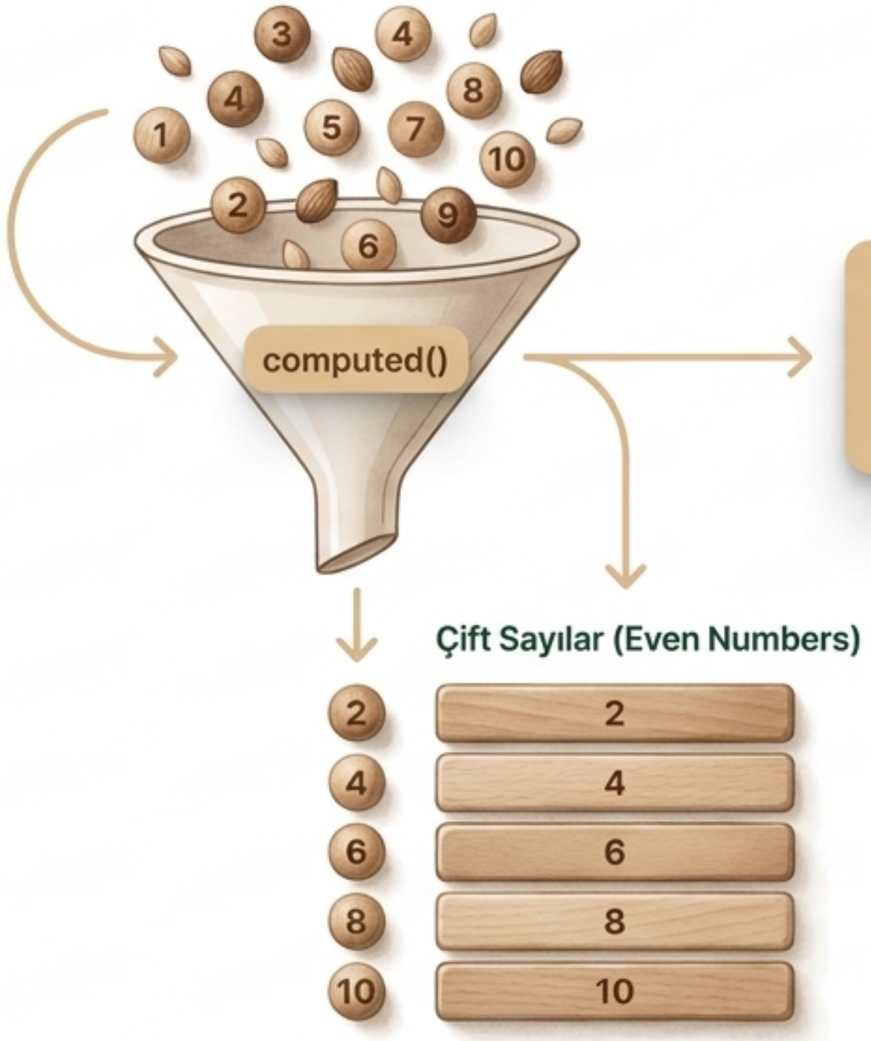
Orijinal diziyi bozmaz, yeni bir dizi döndürür. Eski diziyi yenisiyle değiştirmek gerekir.

`filter()` `concat()` `slice()`

Yeni bir dizi atamak tüm DOM'u sıfırdan çizmez. Vue'nun Sanal DOM algoritması, mevcut öğeleri en yüksek verimle yeniden kullanacak kadar akıllıdır.

Gemini Notebook

Orijinal veriyi bozmadan listeleri filtrelemek için hesaplanmış özellikleri kullanın.



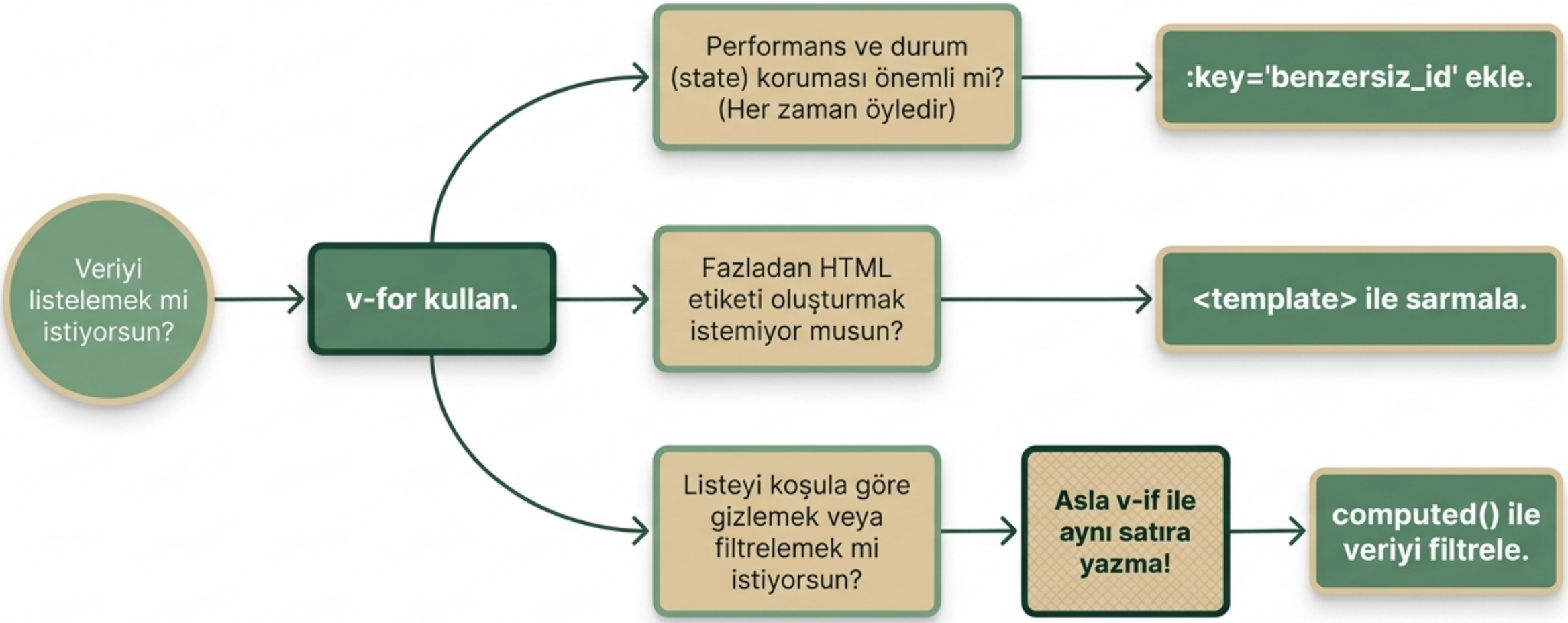

```
const evenNumbers = computed(() => {
  return numbers.value.filter((n) => n % 2 === 0)
})
```

Alternatif: Metot Kullanımı

İç içe döngülerde computed uygun olmadığında, v-for içinde `even(numbers)` gibi bir metot çağrısı kullanın.

Gemini Notebook

Liste render etme mimarisi için karar ağacı.



Gemini Notebook

3. Bölüm: İzleyiciler (Watchers)

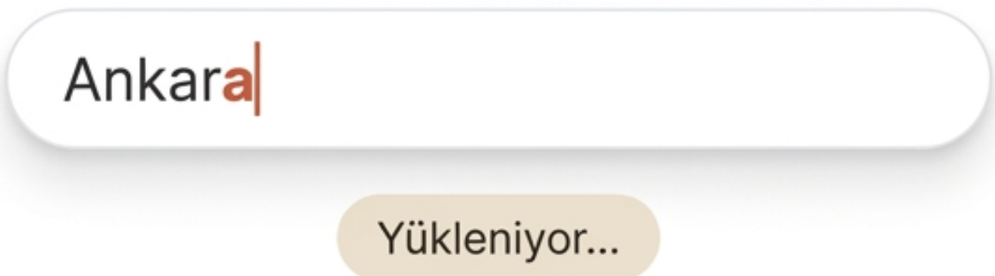
Etki ve Tepki

Computed (Hesaplanmış Özellikler) yeni veri üretir.
Watcher (İzleyiciler) ise dış dünya (API, DOM) ile konuşur.

Gemini Notebook

Değişimi Dinlemek

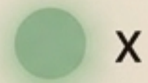
```
watch(sehir, async (yeniSehir) => {
  yukleniyor.value = true
  veri = await fetchWeather(yeniSehir)
  yukleniyor.value = false
})
```



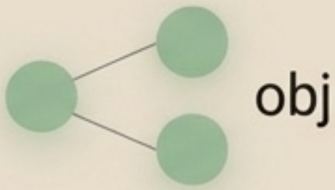
Gemini Notebook

Gözlem Kapsamı

Tek Ref



Reaktif Nesne



Getter Fonksiyonu

Çoklu Kaynak Dizisi

❌ **YANLIŞ:** watch(obj.count, ...) (Sadece değer iletir)

✅ **DOĞRU:** watch(() => obj.count, ...) (Getter ile referansı korur)

Gemini Notebook

Modlar ve Ayarlar

ON

Derin (deep: true)

İç içe geçmiş tüm özellikleri tara. (⚠️ Büyük verilerde performansa dikkat!)

ON

İstekli (immediate: true)

Değişimi bekleme. Sayfa açılır açılmaz fonksiyonu ilk kez hemen çalıştır.

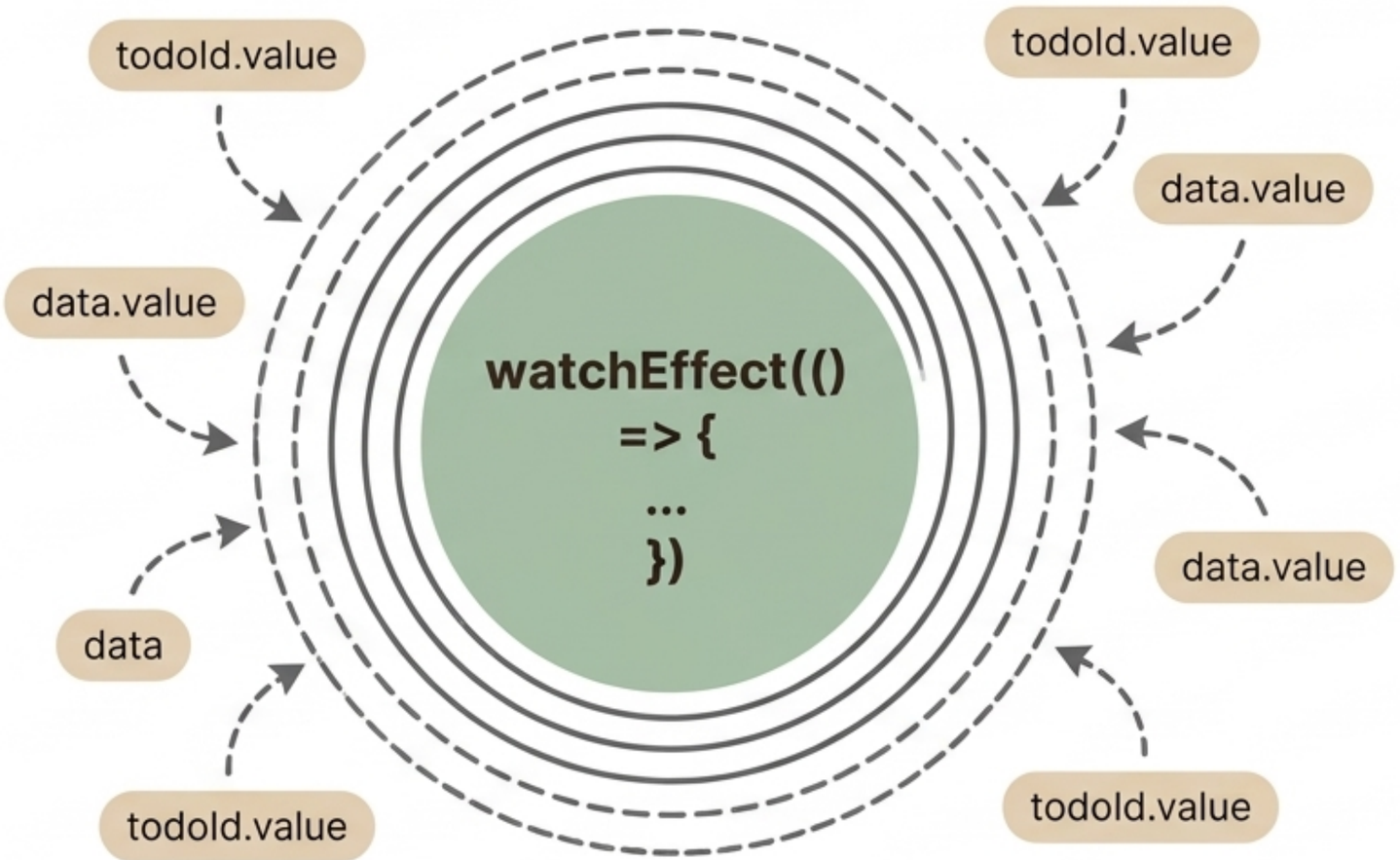
ON

Tek Seferlik (once: true)

Sadece ilk değişimde tetiklen, ardından kendini otomatik imha et (Vue 3.4+).

Gemini Notebook

Otomatik Pilot: watchEffect()



Bağımlılıkları tek tek yazmanıza gerek yok. İçeride hangi reaktif veriyi okursanız, onu otomatik takip eder.

Not: Her zaman ‘İstekli’ (Immediate) olarak başlar.

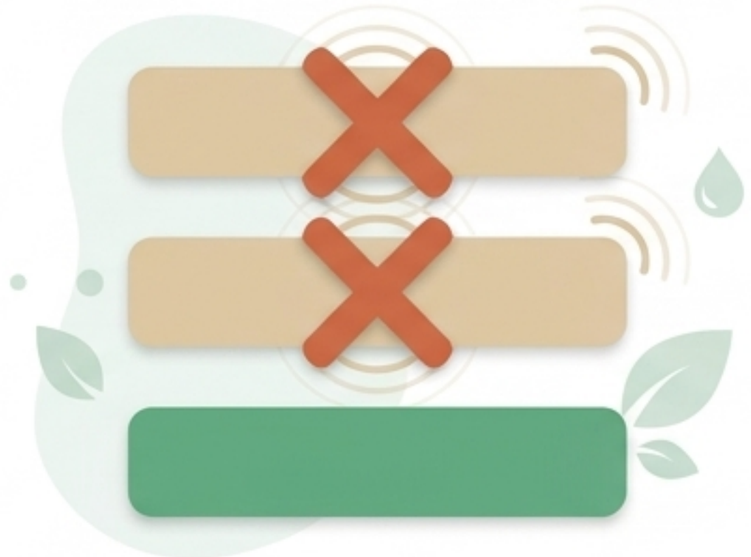
Gemini Notebook

Hangisini Seçmeliyim?

	watch	watchEffect
Bağımlılık Takibi	Yalnızca belirtilenleri izler	İçeride okunan her şeyi otomatik izler
Eski/Yeni Değer Ayrımı	Sağlar (newValue, oldValue) ✅	Sağlamaz ❌
İlk Çalışma	Varsayılan tembeldir (Lazy)	Varsayılan hemen çalışır (Immediate)
Kullanım Amacı	Değişim olduğunda tam kontrol	Birden çok veriye dayalı otomatik senkronizasyon

Gemini Notebook

Temizlik ve Zamanlama Kontrolü



Temizlik

`onCleanup(() ⇒ controller.abort())`

Yeni istek başladığında eskisini iptal et.

Zamanlama

Gemini Notebook

Altın Kurallar

1

Veriden yola çıkıp yeni bir veri üretiyorsan -> **Computed** kullan.

2

Veri değiştiğinde dışarıya (API, DOM) bir iş yaptıracaksan -> **Watch** / **watchEffect** kullan.

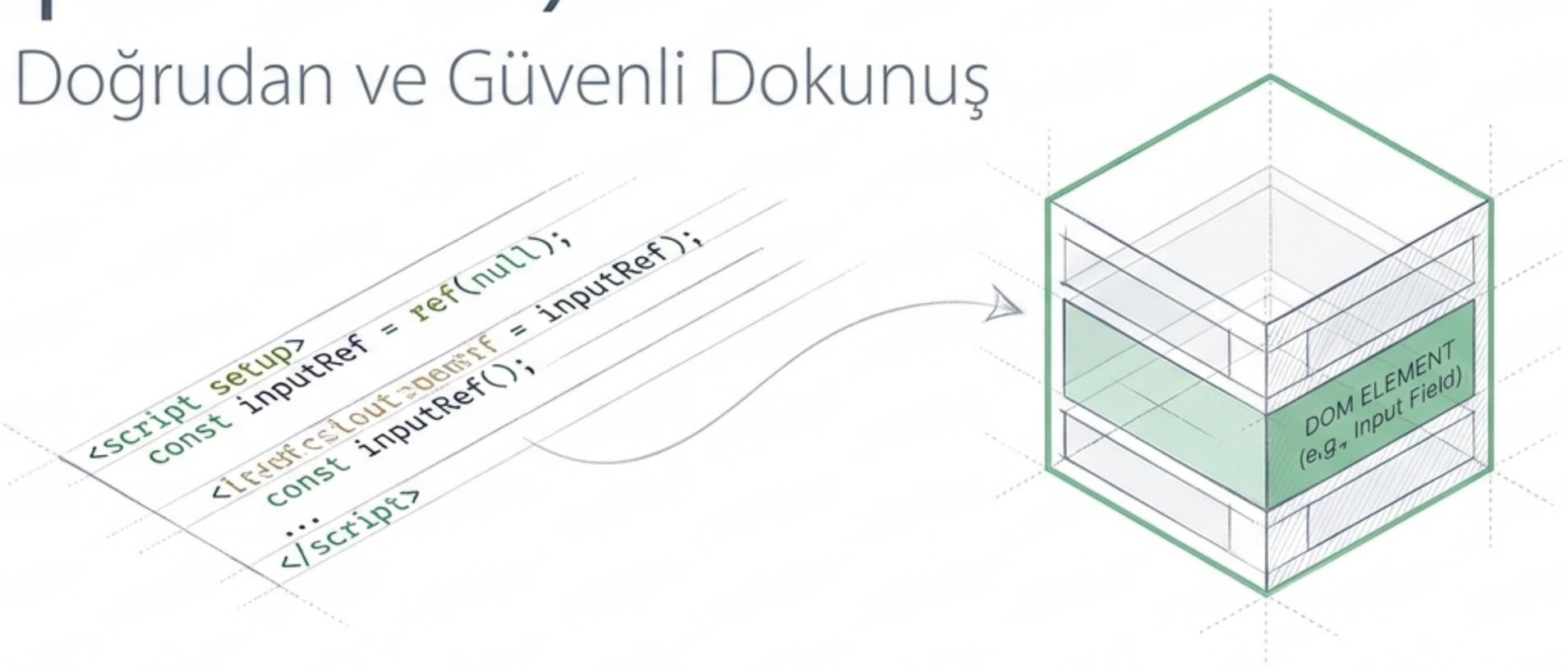
3

Manuel başlattığın izleyicilerin bellekte sızıntı yapmaması için işin bitince **unwatch()** ile durdur.

Gemini Notebook

Vue.js'te Şablon Referansları (Template Refs)

DOM'a Doğrudan ve Güvenli Dokunuş

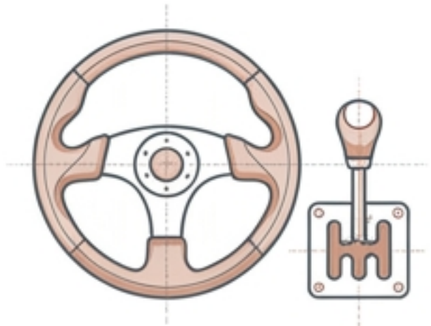


Gemini Notebook

4. Bölüm: Şablon Referansları (Template Refs)

DOM'u Vue Yönetir

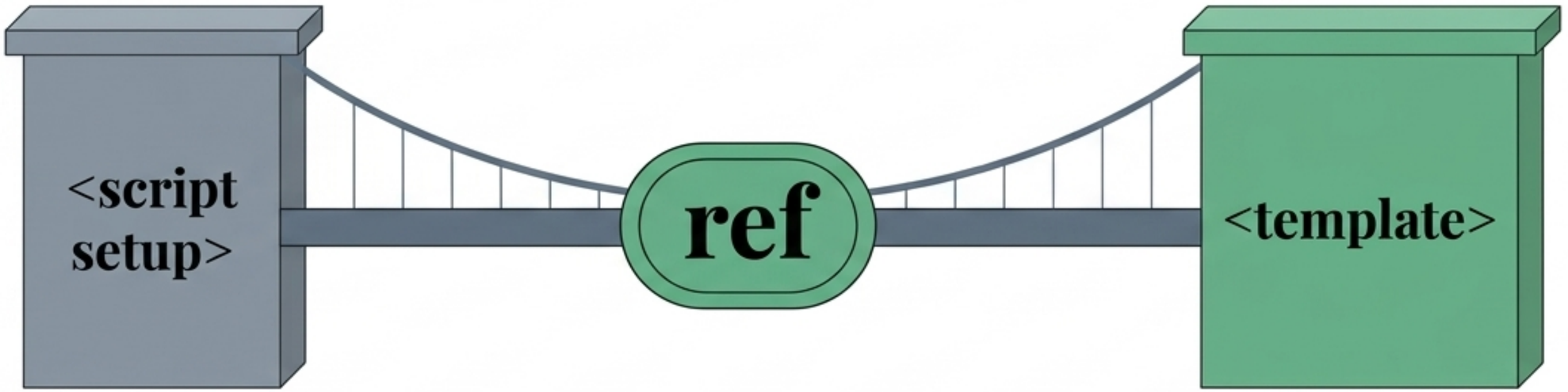
Vue'nun bildirimsel render modeli, veri ve arayüzü otomatik olarak eşitler. Çoğu zaman DOM'a dokunmanız gerekmez.



Bazen Kontrol Sizde Olmalıdır

Bir metin kutusuna programatik odaklanmak (focus) veya bir öğenin boyutlarını hassas ölçmek gibi ham DOM erişimi gereken anlarda manuel bir köprüye ihtiyaç duyarız.

Gemini Notebook



ref, şablon render edildikten sonra belirli bir DOM öğesine veya alt bileşene doğrudan erişim sağlayan özel bir özniteliktir. key özniteliğine benzer şekilde çalışır.

```
<input ref="input">
```

Gemini Notebook

Template Refs: Sözdizimi Karşılaştırması

Klasik Sözdizimi (Vue 3.0 - 3.4)	Modern Sözdizimi (Vue 3.5+)
Kural: Tanımlanan değişkenin adı, şablondaki ref değeri ile tamamen aynı olmak zorundadır.	Kural: useTemplateRef kullanılır. Değişken adı bağımsızdır, isim zorunluluğu ortadan kalkar ve daha esnektir.
`const input = ref(null)` `<input ref="input" />`	`const myElement = useTemplateRef('my-input')` `<input ref="my-input" />`

Gemini Notebook

En Önemli Kural: Önce İnşaat, Sonra Ziyaret

setup() bloğu

Durum:
Bina henüz inşa edilmedi. İlk render gerçekleşmedi.

Referans değeri null'dır.

onMounted() aşaması

Durum:
Bileşen DOM'a başarıyla bağlandı.

Referans artık gerçek DOM öğesine dönüşmüştür.

Uyarı: Bir şablon referansına asla setup bloğunun en üst seviyesinde erişmeye çalışmayın. Sadece onMounted sonrasında kullanın!

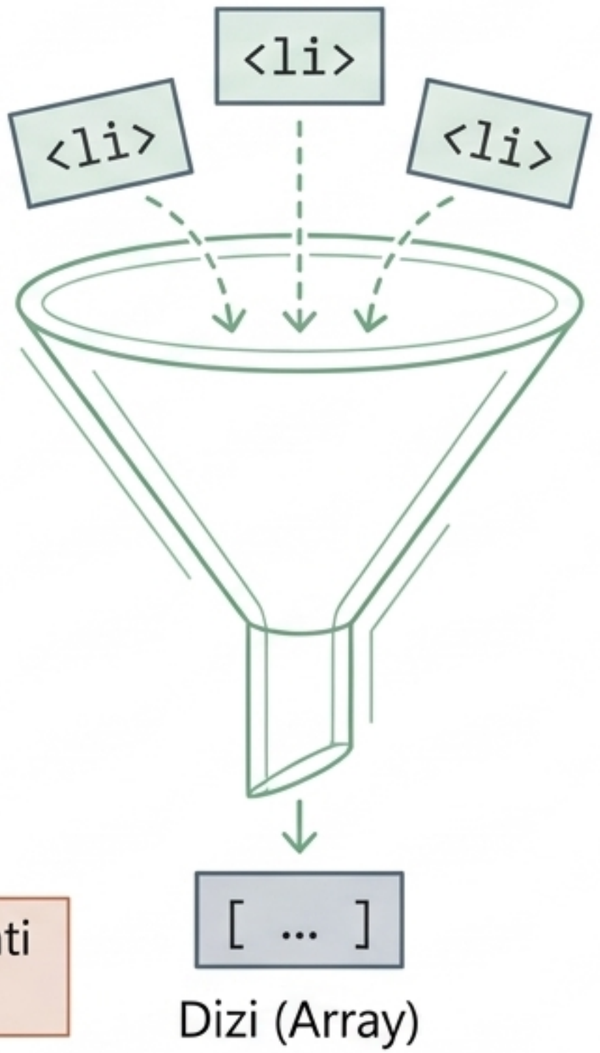
Gemini Notebook

Koleksiyonlar ve v-for Döngüleri

v-for içinde ref özniteliği kullanıldığında, Vue karşılık gelen referans değerini tek bir öğe değil, DOM öğeleriyle doldurulmuş bir Dizi (Array) haline getirir.

```
<li v-for="item in list" ref="itemRefs">
```

`itemRefs.value` artık bir dizidir.



Kritik Not: DOM içindeki sıralama, kaynak dizinin sırasını her zaman garanti etmeyebilir. Güvenli eşleşme için kimliklere (ID) göre işlem yapılmalıdır.

Gemini Notebook

İleri Düzey: Fonksiyon Referansları (Function Refs)

Dinamik bir dize (string) anahtarı atamak yerine, **:ref** özniteliğine doğrudan bir fonksiyon iletebilirsiniz.

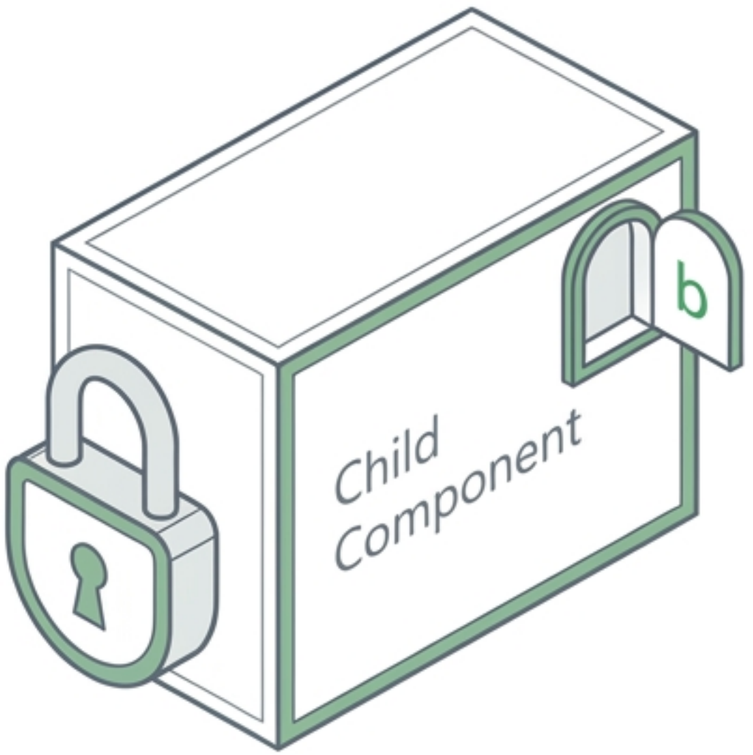
- Bu fonksiyon, bileşenin her güncellemesinde çağrılır ve öğenin referansını (el) parametre olarak alır.
- Öğe DOM'dan söküldüğünde (unmounted), fonksiyon null parametresiyle tekrar çağrılır.

```
<input :ref="(el) => { myDynamicVar = el }">
```

Gemini Notebook

Alt Bileşenler ve Kilitli Kapılar (defineExpose)

ref özniteliği bir alt bileşene yerleştirilirse, DOM öğesine değil Bileşen Örneğine (Instance) ulaşırsınız. Ancak <script setup> varsayılan olarak kapalıdır (closed).

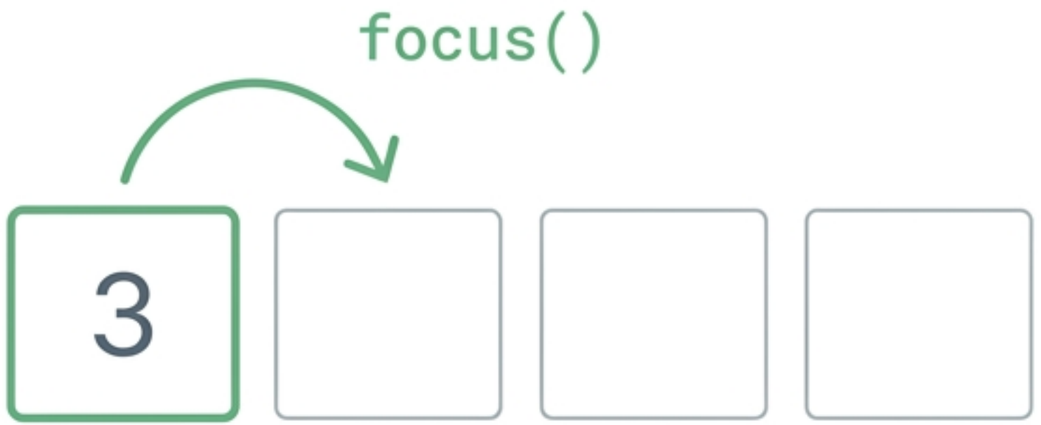


Ebeveyn bileşenin, alt bileşendeki bir değişkene erişebilmesi için alt bileşenin bunu defineExpose makrosu ile açıkça dışarı aktarması zorunludur.

```
defineExpose({ b }) // b dışa açık, diğer her şey gizli
```

Gemini Notebook

Gerçek Dünyada Kullanım: Otomatik Odaklanma (Auto-Focus)



Senaryo

Kullanıcı OTP kodunun ilk rakamını girdiğinde, sistemin otomatik olarak onu ikinci kutucuğa taşıması (focus) gerekir.

Neden ref kullanıyoruz?

Odaklanma işlemi ham bir tarayıcı/DOM API özelliğidir. Vue'nun veri reaktivitesi bunu doğrudan yapamaz.

Çözüm

v-for ile üretilen kutucuklar ref dizisinde tutulur. İlk giriş algılandığında **inputRefs.value[1].focus()** programatik olarak tetiklenir.

Gemini Notebook

Şablon Referansları: Akılda Kalması Gerekenler

Amaç

Ham DOM öğelerine veya Alt Bileşen örneklerine programatik müdahale etmek (focus, ölçüm vb.).

Kritik Zamanlama

Sadece onMounted aşamasından sonra kullanın. İlk kurulum (setup) anında her şey **null**'dır. İnşaat bitmeden içeri girilmez.

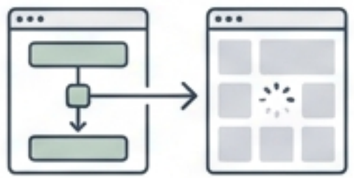
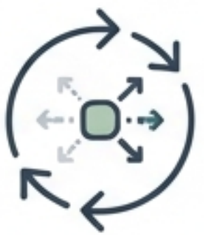
Güvenlik ve Kapsülleme

Alt bileşenler varsayılan olarak **izole** ve **kapalı**dır. İçeriye kontrollü erişim sağlamak için **defineExpose** kullanmak şarttır.

Gemini Notebook

5. Bölüm: Yaşam Döngüsü Kancaları (Lifecycle Hooks)

Bileşenler Tıpkı Doğadaki Canlılar Gibi Doğar, Büyür ve Yok Olurlar

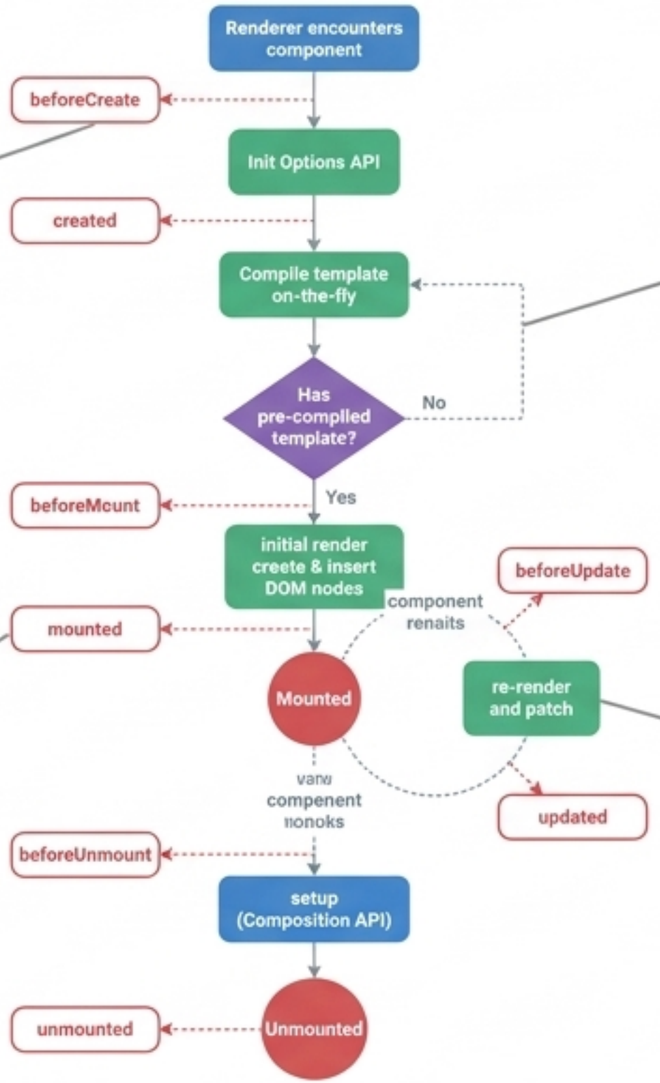
Doğa	Yazılım (Vue)
Tohumun toprağa düşmesi (Filizlenme).	 DOM'a bağlanma ve ilk veriyi çekme (Mounting).
Yaprak açıp güneşe adapte olması (Büyüme).	 Reaktif verilere göre şekil değiştirme (Updating).
Kuruyup toprağa karışması (Solma).	Döngünün sonu ve etrafı temizleme (Unmounting).

Yaşam döngüsü kancaları (lifecycle hooks), geliştiriciye bu evrelerin tam geçiş anlarında koda müdahale etme yetkisi verir.

Gemini Notebook

Vue Yaşam Döngüsü Haritasının Tamamı

1. Başlatma: Veri gözleminin kurulması ve Option/Composition API hazırlığı.



2. Derleme: Şablonun (template) anında derlenmesi.

3. Montaj (Mounting): Örneği gerçek tarayıcı DOM'una bağlama işlemi.

4. Reaktivite Döngüsü: Veri değiştiğinde tetiklenen re-render ve yama (patch) mekanizması.

Gemini Notebook

Sahneye Çıkış ve Toprağa Tutunma: onMounted

Bileşenin gerçek DOM'a yerleştiği ve ilk render'ını tamamladığı an.

Bitkinin toprağa kök salıp ilk suyunu çekmesi.

DOM odaklanması (focus()) • API'den ilk veriyi çekme • Canvas/Harita başlatma

```
<script setup>
import { onMounted } from 'vue'

onMounted(() => {
  console.log(`Bileşen başarıyla
DOM'a bağlandı.`)
  // API çağrıları burada yapılır
})

</script>
```

Gemini Notebook

Çevresel Değişimlere Adaptasyon: onUpdated

Reaktif veri değişikliği Sanal DOM güncellemesine yol açıp, gerçek DOM yenilendiğinde çalışır. Güncelleme sonrası DOM boyutu hesaplamaları için idealdir.

Asla onUpdated içinde izlenen state'i (reaktif veriyi) değiştirmeyin!

Bu durum uygulamanın sonsuz bir güncelleme döngüsüne (infinite loop) girmesine ve çökmesine neden olur.

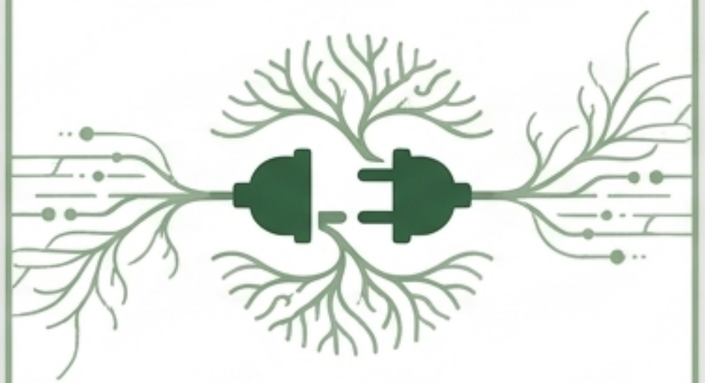
Gemini Notebook

Doğada İz Bırakmadan Ayrılmak: onUnmounted

Bileşen DOM'dan söküldü, tüm alt bileşenler ve izleyiciler kapatıldı. Artık temizlik zamanı.

Zamanlayıcılar

Açık kalan setInterval veya setTimeout fonksiyonlarını temizleme (clearInterval).



Bağlantılar

Aktif WebSocket bağlantılarını güvenli bir şekilde kapatma ve sökme.

Dinleyiciler

Tarayıcı penceresine (window/document) eklenen global olay dinleyicilerini (event listeners) kaldırma.

Gemini Notebook

Kritik Çerçeve Kuralı: Kancalar Senkron Kaydedilmelidir

onMounted çağrıldığında Vue, bu fonksiyonu o anda aktif olan bileşen örneğiyle otomatik ilişkilendirir. Eğer asenkron bir çağrı yaparsanız, Vue ipin ucunu kaçırır ve hangi bileşene ait olduğunu bilemez.

Doğrudan ve Senkron

```
onMounted(() => {
  /* Başarılı. Vue örneği tanır. */
})
```

Çağrı yığını senkron olduğu için Vue örneği tanır ve kayıt başarılı olur.

Asenkron (Kayıp Referans)

```
setTimeout(() => {
  onMounted(() => { /* ÇALIŞMAZ! */ })
}, 100)
```

Zamanlayıcı bitip fonksiyon çalıştığında, Vue artık fonksiyonun hangi bileşene ait olduğunu bilemez.

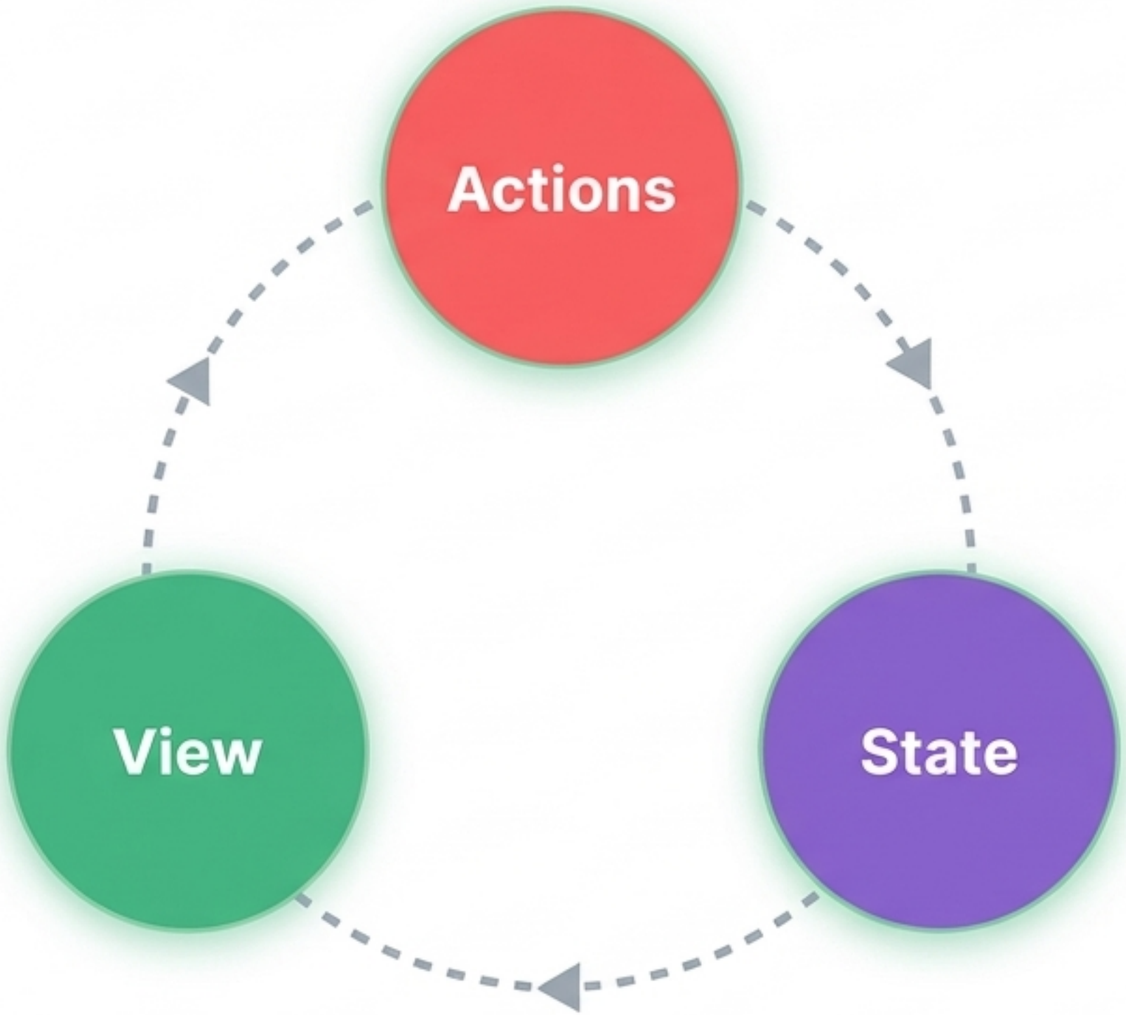
Gemini Notebook

Geliştiricinin Anatomik Kopya Kağıdı

Kanca Adı	Ne Zaman Çalışır?	Kullanım Amacı
onBeforeMount	Şablon derlendi, henüz DOM'a yerleşmedi	Son veri kontrolleri.
onMounted	Bileşen gerçek DOM'a yerleştirildi	DOM odaklanması, API'den ilk veri çekimi, Canvas başlatma.
onBeforeUpdate	Reaktif veri değişti, Sanal DOM henüz güncellenmedi	Güncelleme öncesi mevcut DOM durumunu okuma.
onUpdated	DOM Sanal DOM yama işlemiyle güncellendi	Güncelleme sonrası DOM boyutu hesaplamaları.
onBeforeUnmount	Bileşen sökülmek üzeredir, hala çalışır durumda	Temizlik hazırlıkları.
onUnmounted	Bileşen söküldü, her şey kapandı	Zamanlayıcıları, dinleyicileri ve soketleri tamamen temizleme.

Gemini Notebook

Tek Yönlü Veri Akışı (One-Way Data Flow)



Durum (State): Uygulamayı yönlendiren gerçeklik kaynağı.

```
const count = ref(0)
```

Görünüm (View): Durumun şablon üzerindeki yansıması.

```
<template>{{ count }}</template>
```

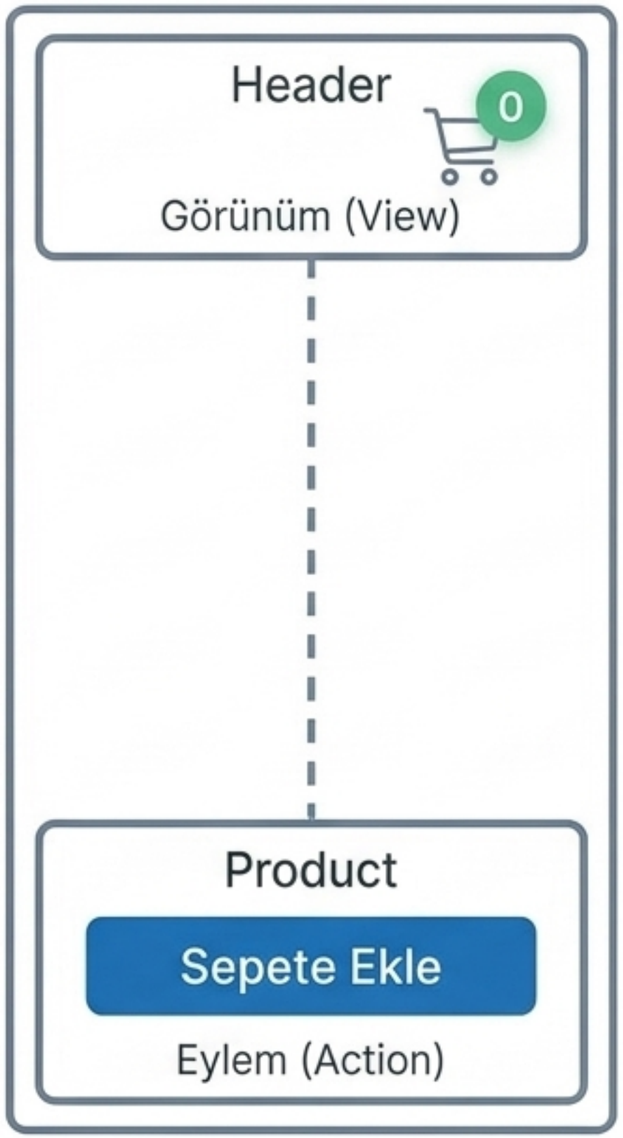
Eylemler (Actions): Kullanıcı etkileşimiyle durumu değiştiren fonksiyonlar.

```
function increment()
```

Her Vue bileşeni kendi reaktif durumunu bağımsız olarak mükemmel bir şekilde yönetir.

Gemini Notebook

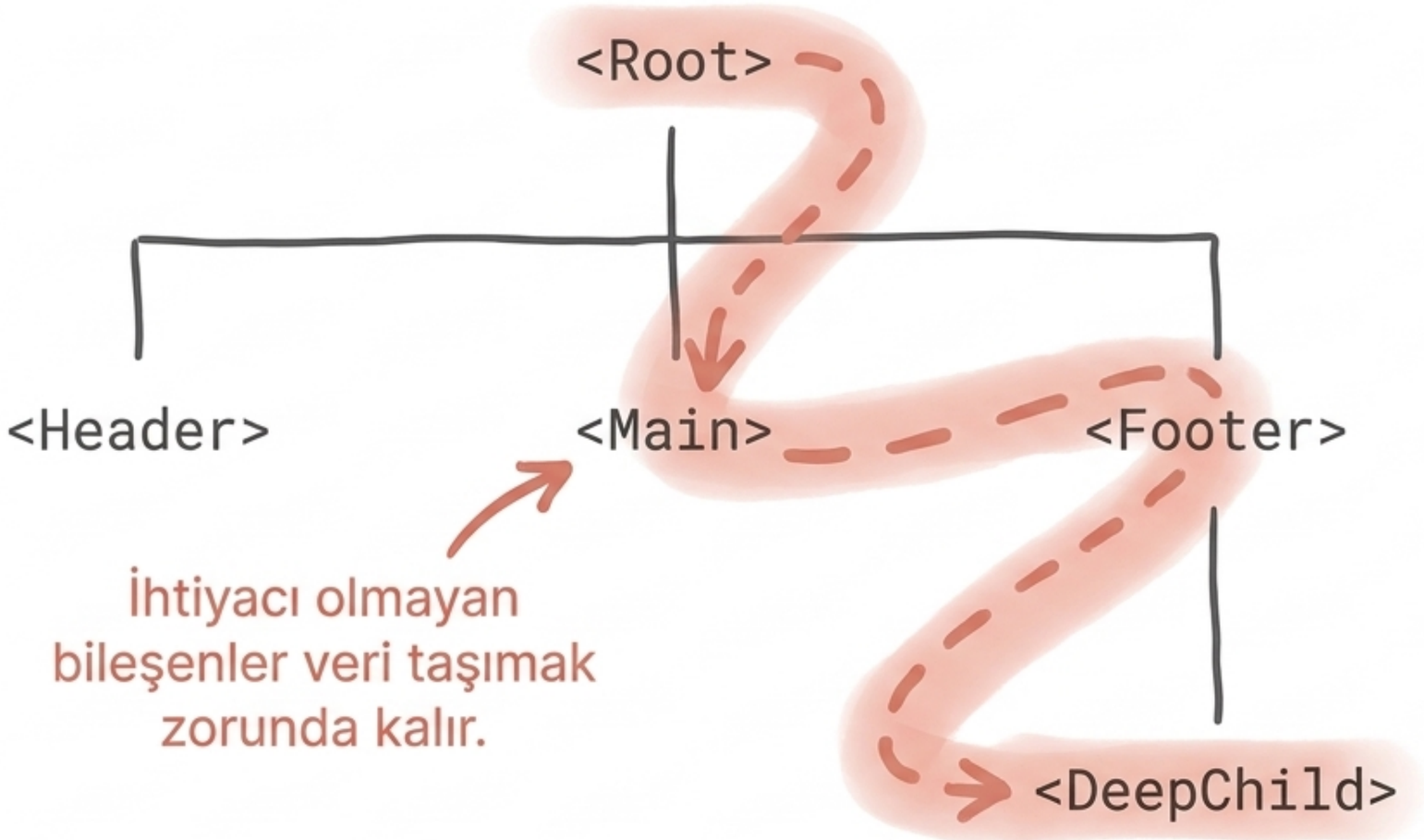
Gerçek Dünya Problemi: Alışveriş Sepeti



Alttaki ürün bileşeninde gerçekleşen bir eylem, en üstteki başlık bileşenindeki durumu (sepet sayısını) nasıl günceller?

Gemini Notebook

Prop Drilling Krizi

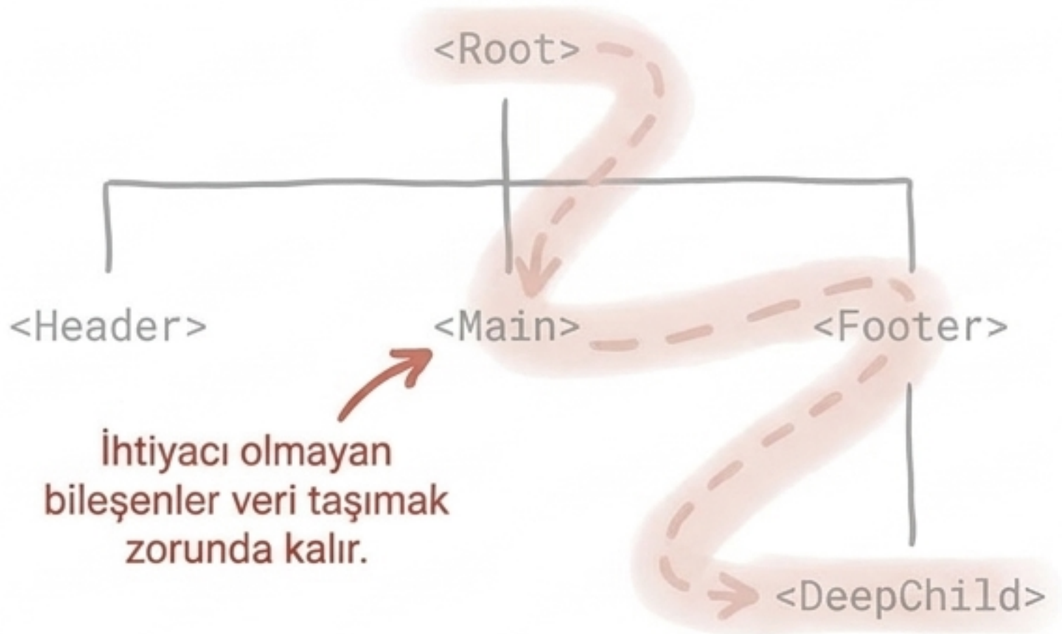


- Paylaşılan durumu yukarı taşımak (lifting state up) ve props olarak aşağı iletmek derin hiyerarşilerde hantal hale gelir.
- Alternatif geçici çözümler ($emit veya doğrudan referanslar) kırılgandır ve kodun bakımını imkansız hale getirir.

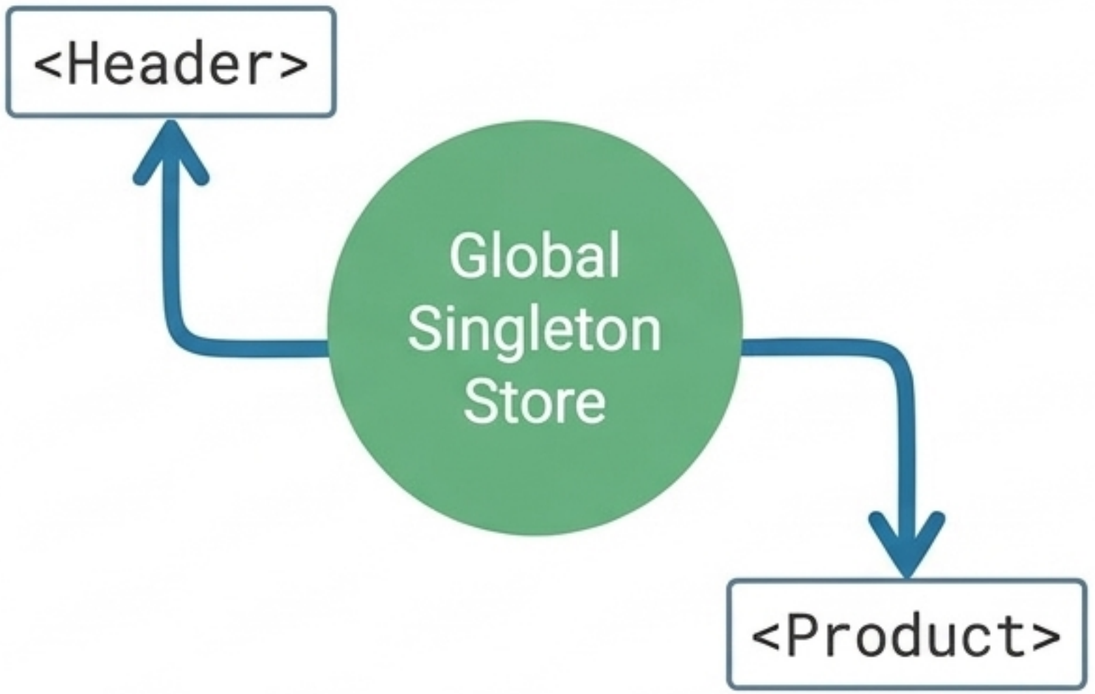
Gemini Notebook

Çözüm: Durumu Dışarı Aktarmak

Sorun



Çözüm



Paylaşılan durumu bileşen ağacının dışına çıkarıp küresel bir tekil merkezde yönettiğimizde, tüm bileşen ağacımız dev bir görünüm haline gelir.

Gemini Notebook

Reaktivite API'si ile Basit Çözüm

store.js

```
export const store = reactive({
  count: 0,

  increment() {
    this.count++
  }
})
```

Bileşenler (ComponentA & ComponentB)

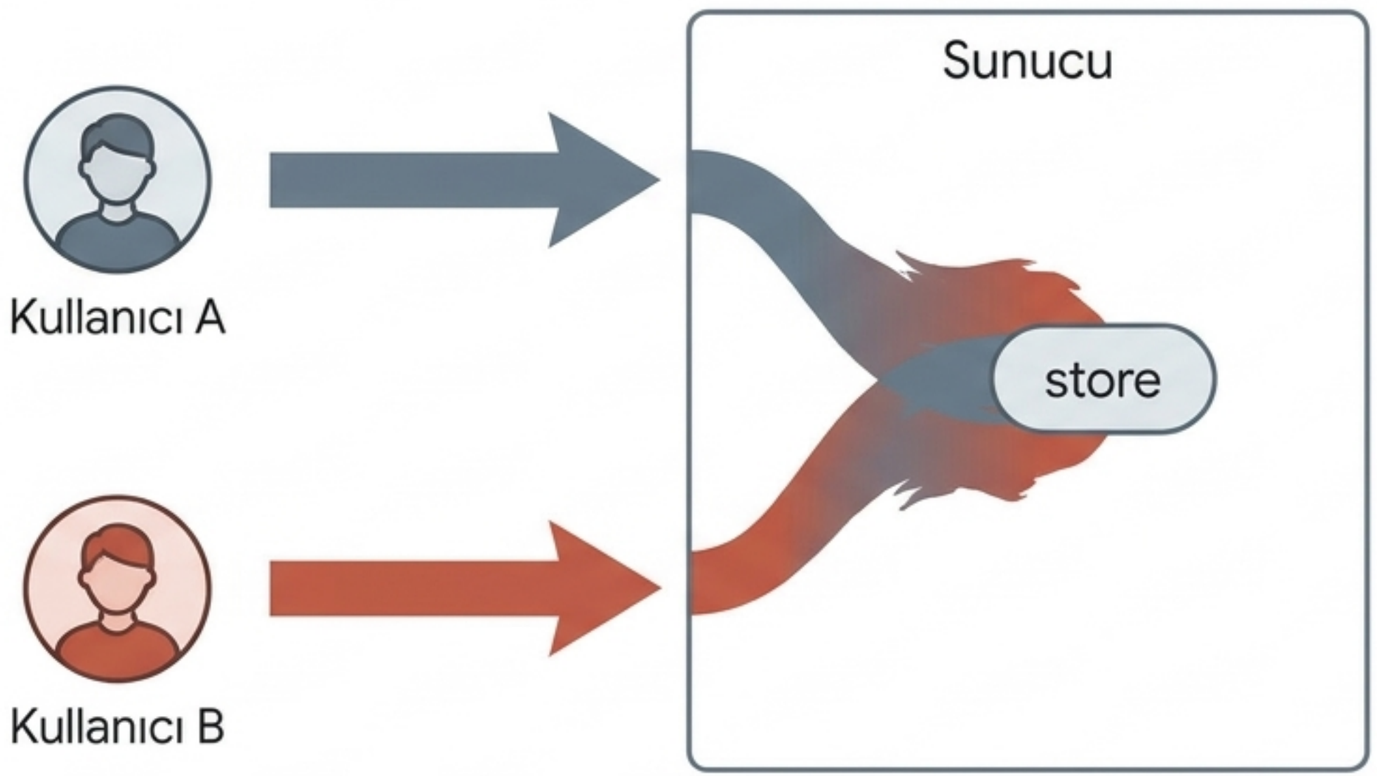
```
<script setup>
import { store } from './store.js'
</script>

<template>
  <button @click='store.increment()'>
    Değer: {{ store.count }}
  </button>
</template>
```

ⓘ Durum mutasyon mantığını, durumun kendisi gibi merkezileştirmek en iyi pratiktir.

Gemini Notebook

Basit Çözümün Sınırı: Sunucu Taraflı Render (SSR)



- Basit singleton kalıbı, store'un gelen birden çok istek arasında paylaşılması nedeniyle istekler arası durum kirlenmesine (cross-request state pollution) yol açar.
- Üretim düzeyindeki büyük projelerde daha sağlam bir altyapıya ihtiyaç vardır.

Gemini Notebook

Modern Vue Standardı: Pinia

Zaman Yolculuğu (Time-Travel Debugging)

Resmi Kütüphane

Vue'nun resmi durum yönetimi kütüphanesidir.

SSR Uyumlu

Kusursuz Sunucu Taraflı Render (SSR) desteği sunar.

Vuex'e Ne Oldu?

Vuex artık bakım modundadır (maintenance mode). Yeni projeler için kesinlikle Pinia önerilir.

Gemini Notebook

Neden Pinia'yı Tercih Etmeliyiz?

1. Daha Az Kod ve Tören

Karmaşık mutations yapısı tamamen ortadan kalkmıştır. Doğrudan state ve actions ile temiz ve sade bir şekilde çalışılır.

2. Composition API Doğallığı

Modern <script setup> sözdizimiyle tamamen doğal şekilde entegre olan yenilikçi Setup Store standardı sunar.

3. Mükemmel TypeScript Desteği

Ekstra karmaşık arayüzler (interfaces) tanımlamaya gerek kalmadan, yazılan koddan otomatik tip çıkarımı (type inference) sağlar.

Gemini Notebook

Hangi Yöntemi Seçmeliyiz?

Kriter	Yerel Durum (ref)	Basit Küresel (reactive)	Pinia
Kurulum Karmaşıklığı	Yok	Çok Düşük	Düşük (Kütüphane gerektirir)
Uygun Proje Boyutu	Tekil Bileşen	Küçük / Hobi Projeleri	Kurumsal / Büyük Ölçekli
DevTools Desteği	Sınırlı	❌ Yok	✅ Tam Destek (Zaman Yolculuğu)
SSR Güvenliği	✅ Güvenli	❌ Riskli (Kirlenme)	✅ Güvenli (Tam Uyumlu)

Doğru mimari, projenin ölçeğine göre doğru aracı seçmektir.

Gemini Notebook